

Original Idea:

The original idea behind this project was to combine my knowledge of programming and security to make something public so employers can contact me. I wanted to make my own small network which would have consisted of a web server, firewall, a home lab, and an access point for all my smaller devices. This would have allowed me to make an internal network zone and a DMZ (Demilitarized Zone). However while doing this project I ran into problems, mostly with the firewall, which made me completely redesign how this project was going to work.

Things I Knew Beforehand:

- LAN(Local Area Network): Spans a local area (office or home) to share resources (internet connections, files, etc.)
- WAN(Wide Area Network): Connects multiple LANs across a large area (region, globally, etc.)
- VLAN(Virtual Local Area Network): Creates multiple virtual networks on a single network, usually done with a managed switch
- IP Addresses: Identifies each computer using a string of numbers to communicate on a network
- Port Numbers(80, 443, 22): Port 80 is for HTTP. Port 443 is for HTTPS. Port 22 is for SSH
- Inbound and Outbound traffic: Data/requests entering a device/network and data/requests leaving a device/network to an external destination
- Eth0 and Eth1: Eth0 is WAN and Eth1 is LAN
- Internal network zone: A network that's private and may contain sensitive data, and should only be accessible to authorized internal users/devices
- DMZ(Demilitarized Zone): A subnetwork that acts as a buffer between an internal network, trusted network, and untrusted external network (ex Internet)
- Linux commands: It's a fundamental way to interact with a Linux based system

Development Process:

Attempt 1 (iptables, and dnsmasq):

Research Done During Initial Set Up:

iptables

- A linux firewall utility for filtering network packets, allowing the configuration of rules for packet filtering, network address translation (NAT), and packet mangling
 - Packet filtering: a network security technique used to control traffic by examining the headers of data packets and allowing or blocking them based on predefined rules such as source/destination IP addresses and port numbers
 - Network address translation(NAT): a technique that allows multiple devices on a private local network to share a single public IP address to access the internet, conserving public IPv4 addresses and providing security by hiding the internal network's IP structure from external networks
 - Packet mangling: the intentional modification of data within an IP packet's header distinct from malicious attacks

dnsmasq

- A lightweight, versatile service that provides DNS caching, DHCP, and TFTP services, primarily for small networks like home/office
 - DHCP (Dynamic Host Configuration Protocol): a network protocol that automates the assignment of IP addresses and other configuration details (like subnet masks, default gateways, and DNS server addresses) to devices on a network
 - Subnet masks: a 32-bit number that divides an IP address into a network portion and a host portion, identifying the network and the device on it respectively. It allows network devices to determine if a destination IP address is on the same local network or needs to be routed elsewhere facilitating efficient network traffic flow, security, and organization
 - TFTP (Trivial File Transfer Protocol): a simple lightweight and connectionless protocol for transferring files
 - DNSSEC (Domain Name System Security Extensions): adds cryptographic security to the DNS protocol by verifying data origin and integrity, preventing DNS spoofing and man in the middle attacks by establishing a chain of trust
 - DNS (Domain Name Service) protocol: is a set of rules and processes that translate human readable domain names into machine readable IP addresses enabling users to access internet resources
 - Spoofing: a cyberattack in which a perpetrator disguises themselves as a trusted individual business or entity to trick victims into revealing confidential information or downloading malicious software
 - Man in the middle: a cyberattack where an attacker secretly intercepts and potentially alters communications between 2 unsuspecting parties posing as both a participant to one or both parties to gain unauthorized access to data or manipulate the communication

- It acts as a DNS server, caching frequently accessed domain names, and as a DHCP server, automatically assigning IP address to devices on the network

Problems I ran into:

1.Subnet Mismatches:

What happened:

On the "dhcpd.conf" file eth0 was assigned to a static IP on the 192.168.50.0/24 subnet. But on the "dnsmasq.conf" the DHCP was set to "192.168.10 - 192.168.60.100". So when dnsmasq started it asked if any interfaces had an IP in 192.168.50.0/24, and the answer was no.

Why this was a problem:

DHCP only hands out addresses on the subnet that matches the serving interface. And because the Pi wasn't on the "192.168.60.0/24" network the client never received leases. ie this broke connectivity for clients relying on the Pi for an IP.

How I tried to fix it:

I went into the "dhcpd.conf" file and changed the interface IPs to align with my DHCP range. I restarted the dnsmasq and re-tested with the ipconfig getpacket on my Mac to see if my Pi was giving leases.

Lesson I learned:

The ranges on DHCP and IPs have to match

2.DHCP conflicts:

What happened:

On my router it has its own DHCP service enabled on 192.168.8.0/24. However at the same time my Pi tried to run DHCP for another subnet. So some devices ended up requesting DHCP and got answers from the Pi and or the router. Also when I called it a day, my laptop and my phone started to act weird.

Why was this a problem:

Receiving multiple DHCPs at the same time is very confusing for devices. Those devices would flip between subnets or just get invalid configurations. My laptop and phone started to act weird because it was dropping the connection because it was failing to reach my router's DNS.

How I tried to fix it:

I went into my routers settings to disable my DHCP, so my firewall can handle it (ie only making one DHCP server for the subnet). I then monitored the dnsmasq logs to confirm leases were issued.

Lesson I learned:

Only one DHCP can operate per subnet, ie either the router or my firewall not both

3.Single Interface Use:

What happened:

The Pi was only using Eth0 and multiple subnets were bound to this one interface. I.e. the Pi didn't know what was LAN and which was WAN

Why was this a problem:

The firewall needs a way to separate the trusted traffic(inside) from the untrusted traffic(outside). And because there was only one interface the NAT and firewall got confused. It sometimes allowed traffic to bypass the firewall and other times it would completely block everything.

How I tried to fix it:

I went back and added a second interface, Eth1, with a clear LAN IP. I set Eth0 as WAN, which was connected to my router, and then set Eth1 as LAN, which was connected to my switch. I also went and adjusted my iptables rules to specifically forward my Eth1 → Eth0 for outbound traffic

Lesson I learned:

The firewall has to have 2 distinct interfaces because otherwise the traffic classification fails

4. Service Persistence:

What happened:

The rules I set for my firewall, with iptables, would work until a reboot then after that reboot all my rules would be gone. (i.e. the rules that I would set with iptables would be saved when I would set them but when rebooted it would lose everything)

Why was this a problem:

This would defeat the whole purpose of building a firewall because after a crash or a power cycle it wouldn't function properly.

How I tried to fix it:

I saved the rules with "iptables-save > /etc/iptables/rules.v4". I also looked into wrapping them in rc.local and or a systemd to reload it automatically. I then looked into ufw for an easier persistence.

Lesson I learned:

Always make sure that the firewall rules save and are persistent because otherwise you're only protected until the next reboot

5. Complexity:

What happened:

My Pi wasn't only doing firewall stuff but it was also running DHCP, DNS forwarding, and static IP assignment all at the same time. So each configuration was dependent on each other, i.e. if one wasn't set up correctly the whole thing would break.

Why was this a problem:

This made debugging really hard and it also led to a cycle of changing numbers a lot without solving the root issue.

How I tried to fix it:

I tried disabling DHCP on my router and let the Pi handle all of the IP assignment. I also commented out parts of my "dhcpd.conf" file to reduce overlap. I also attempted to confirm each service separately, I pinged firewall across subnets and tested DHCP lease with "ipconfig getpackets"

Lesson I learned:

To start small by getting the firewall working first then add other things like the DHCP and the DNS.

Attempt 2 (iptables, dnsmasq, and UFW):

Problems I ran into:

1.Outbound Traffic Wouldn't Return(NAT Missing):

What happened:

Devices behind my raspberry pi were able to send out packets to the internet but the responses never came back.

Why was this a problem:

The LAN clients had no working internet and the packets couldn't get routed back.

How I tried to fix it:

I discovered that my NAT masquerading was required for return traffic. I then applied a rule and after running "curl ifconfig.me" showed the WAN IP and that confirmed the proper routing.

Lesson I learned:

Forwarding alone doesn't guarantee the connection and the outbound packets must be masqueraded so the WAN knows how to reply to the LAN devices.

2.Unsure if IPv6 Needed NAT:

What happened:

I wasn't sure if the same NAT masquerade step was needed for IPv6.

Why was this a problem:

If the IPv6 wasn't configured properly devices could fail to connect or be left unprotected.

How I tried to fix it:

Set IPv6=yes in /etc/default/ufw so that way all my firewall rules apply consistently

Lesson I learned:

NAT is only an IPv4 concept, it's because of private address ranges.

3. Forwarding Rules Between LAN and WAN:

What happened:

I needed the traffic from LAN (wlan0) to reach WAN (eth0) and allow for responses back.

Why was this a problem:

Without the forwarding rules LAN devices wouldn't get access through the firewall.

How I tried to fix it:

I ran two different commands with iptables. One that forwards wlan0 to eth0 to accept and the other command to forward eth0 to wlan0 to accept. This allowed LAN → WAN traffic and allowed for WAN → LAN return traffic.

Lesson I learned:

The forwarding rules on my firewall must be both directional and stateful. These rules also must be persistent just like the NAT rules. Also using "Related,Established" ensures only valid reply traffic is allowed back in which helps protect against abuse.

4. Linux Kernel Level:

What happened:

Upon further research I found out I was only adding security to the kernel level.

Why was this a problem:

This was a major problem because it wasn't secure enough for today's technology. It was a single point of failure that could have been compromised by a multitude of vulnerabilities.

How I tried to fix it:

I looked into different things like pfSense and IPFire to try to keep it more secure.

Lesson I learned:

Security beyond the kernel level is really important to keep everything protected for myself and other users.

I also followed two different tutorials during this attempt:

SSH Tutorial:

Things I learned:

1. Add a non-root user with sudo privileges
2. Generate SSH key pair with ssh-keygen
3. Copy your public key with ssh-copy-id
4. Disable root login and password authentication
5. Update sshd_config for key-only access
6. Restart SSH and verify your hardened setup

Link to video <https://www.youtube.com/watch?v=8ugcUTNoGj4>

Firewall with UFW

Things I learned:

1. Install and enable UFW
2. Allow SSH, HTTP, and HTTPS traffic
3. Block all other unwanted traffic automatically
4. Verify firewall rules work

Link to video <https://www.youtube.com/watch?v=68GTL7djlMI>

My friend Josh helped me out during this attempt too:

Josh Gordon:

Personal Website: <https://joshg13f.github.io/web/>

LinkedIn: <https://www.linkedin.com/in/josh-gordon-22512622/>

Firewall Rules and Policies:

1. Found that my policies were too restrictive
 - Input and forward were set to drop
 - This prevented incoming and forwarded traffic from being processed

Network Address and Translation(NAT):

1. Even with forwarding enabled the return traffic from the internet (WAN to LAN) wouldn't know how to get back to the originating device on the local network (LAN)
 - Showed me how to set my masquerading properly
 - This changed the source IP address of outbound packets to the IP address of the firewall's external interface.
 - Which allowed packets to be routed correctly

Packet Forwarding and Routing:

1. Configured the forwarding rules necessary for firewall setup
 - Routed the traffic between two different interfaces 1. wireless (wlan0) and 2. wired (eth0)
 - This allowed traffic to forward and return between eth0 and wlan0

IPv4 vs IPv6:

1. Clarified the differences between IPv4 and IPv6
 - NAT is required for IPv4 because of its use of private addresses but not for IPv6 because it uses a different addressing scheme

General Troubleshooting:

1. Verify configuration by checking the NAT chains

- `sudo iptables -t nat -L -n -v`
- 2. Confirmed external IP addresses
 - `curl ifconfig.me`
- 3. Debugging tips
 - For different data streams (NAT, LAN to WAN, and WAN to LAN)
 - Meaning of Postrouting chain

Attempt 3 (IPFire and or PfSense):

1. PfSense

What happened:

Just looked into using PfSense to replace my firewall and router.

Why was this a problem:

Because its not officially supported by Raspberry Pi I wasn't able to use it

How I tried to fix it:

There was no fix for this one

Lesson I learned:

PfSense CE is designed to run on x86-64(Intel/AMD) architecture whereas the Raspberry Pi utilizes ARM processors. So although this idea didn't go very far I thought it was important to point out that I was learning about compatibility issues.

2. IPFire

What happened:

I tried to download IPFire onto my Raspberry Pi SD card and when I got it pulled up on the screen it would just show a blinking “_”.

Why was this a problem:

The software (ie IPFire) itself wouldn't let me use it

How I tried to fix it:

At first I thought I downloaded the wrong image so I tried erasing it and trying to download it again. (I was trying to use the aarch64 flash image)

Lesson I learned:

My Raspberry Pi 4 wasn't recognizing the SD card as bootable. Which meant that the IPFire image I downloaded wasn't actually a Pi ready image even though it was an aarch64. As it turns out it was built for ARM servers (UEFI) and not bare metal Raspberry Pi boot. So the IPFire flash image doesn't include the Pi specific boot files that the Raspberry Pi needed to start up the software. Which explains why I was getting the SD error and the blinking “_”.

Attempt 4 (Firewall: UFW and Cloudflare; Web Server: Nginx):

Process:

What happened:

I found a lot more problems that would have been an issue beside the firewall itself. One of them being the current switch I was using. I was trying to use an unmanaged switch to allow traffic to go through my firewall to my web server. The other problem that I would (maybe) have run into was my ISP port numbers may have not been open for me to use (ports 443, 80, and 22). However I wasn't able to actually see if these ports were open for me or not because I wasn't able to get my firewall to work.

Why was this a problem:

My unmanaged switch was a problem because it doesn't support vlan nor port segmentation. In other words my switch wouldn't have allowed me to create separate isolated networks for different devices to communicate (with my firewall and or my web server). If those ports weren't open for me to use then this would have also been a big issue because users wouldn't have been able to even reach my firewall nor my web server.

How I tried to fix it:

This is where I thought about not even controlling my traffic, with a firewall, and maybe allowing someone else to direct the traffic to me/my web server. So I found cloudflare, which allows me to tunnel my traffic directly into my web server with security from cloudflare itself.

Lesson I learned:

I learned that managed and unmanaged switches are very different. Managed switches offer control, security, and performance optimization features like vlans, port monitoring and QoS (Quality of Service). In other words, managed switches allow you to control the traffic over a network. Whereas unmanaged switches are a plug and play with a fixed configuration. Again in other words unmanaged switches expand the network. I also learned to check with your ISP to see if those ports are even open for you to use to allow traffic/users to visit a website on a server.

What am I currently running on my web server?

- Nginx: Serves as a high performance web server, acts as a reverse proxy, load balancer, and cache to efficiently handle and distribute web traffic for my website
- UFW Firewall: Simplifies the management of iptables firewall rules on linux systems
- SSH: Provides a secure way to access and manage remote computers over an unsecured network

- Cloudflare: Makes websites and applications faster, more secure, and highly available by acting as a content delivery network (CDN), DDoS mitigation service, and reverse proxy
- Others which include: Cron(allows users to automate repetitive tasks by scheduling them to run at specific times or intervals), bluetooth, rsyslog (logging), and avahi-daemon(performs zero configuration networking allowing devices on a local network to automatically discover and communicate with each other without manual configuration)

Tutorials I followed during this attempt:

How to install an Nginx web server on Raspberry Pi:

Things I learned:

- 1.Upgrade/update current packages
- 2.Install and get familiar with files within Nginx

Link to video <https://www.youtube.com/watch?v=ECsQ8jbpMow>

I also learned how to get my website from github into the file system instead of making my website in the file system (not in this tutorial though)

Cloudflare Tunnels: Getting started with domains, DNS, and tunnels

Things I learned:

- 1.Buy a brand new domain name from Prok Bun
- 2.New account with cloudflare
- 3.Domain name pointed to cloudflare account
- 4.Cloudflare tunnels
- 5.Have multiple applications on a single tunnel

Link to video <https://www.youtube.com/watch?v=Q5dG8g4-Sx0>

Commands to see rules and whats running on web server:

Check status:

UFW: `sudo ufw status verbose`

Nginx Web Server: `sudo systemctl status nginx`

Listing Running Services:

`sudo systemctl list-units --type=service --state=running`

Listening Ports:

Shows all TCP and UDP: `sudo ss -tulpn`

Firewall Rules:

`sudo iptables -L -n -v`

Lessons Learned Throughout Entire Project:

- DHCP/IP: DHCP ranges and IP ranges must match
- DHCP Operation: Only one DHCP server per subnet (router or firewall)
- Firewall Setup: Firewalls require two distinct interfaces for proper traffic classification
- Persistence: Always ensure firewall/NAT rules are persistent (save and survive reboot)
- Project Approaches: Start small; get the firewall working before adding DHCP/DNS
- NAT/Forwarding: Forwarding alone is insufficient; outbound packets need masquerading (NAT) for WAN reply
- NAT Concept: NAT is an IPv4 only concept due to private addresses
- Firewall Rules: Forwarding rules must be directional, stateful, and use "Related,Establishes" for protection
- Security/Hardening: Beyond kernel level security is vital for protection
- SSH Hardening: Harden SSH using non root user, key pairs, and disabling password/root login
- UFW/Firewalling: Install and enable UFW (or similar), allow necessary traffic (SSH/HTTPS), and block all else
- PFSense/Pi: Architecture incompatibility (PFSense is x86-64, Pi is ARM)
- Pi/IPFire Boot: Aarch64 server images may lack Raspberry Pi's specific boot files
- Switches: Managed switches offer control, security, and optimization (VLANs, QoS); Unmanaged switches are plug and play network expansion
- ISP Ports: Verify ISP allows required inbound ports for server traffic

Future improvements:

- Download OpenWRT OS on my Raspberry Pi for firewall software
- Get a managed switch for VLAN, port monitoring, and QoS support
- Do more research to see if my ISP has ports 443, 80, and 22 open

By doing these few things I believe I can make a working firewall and direct the traffic myself instead of having someone else do it for me. If I'm able to get a firewall working then I'll need to wire it all up and make another SSH key pair, but that's a given.